Start coding or generate with AI.

```
# ======================================================================
# ===== The Complete and Comprehensive Code for SMS Spam Detection =====
# ======================================================================

# ----------------------------------------------------------
# --- PART 1: TRAINING, STATISTICS, AND VISUALIZATIONS ---
# ----------------------------------------------------------

# 1. Importing necessary libraries
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_ma
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

# 2. Download NLTK data (will run silently in the background)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

# 3. Load and Prepare the Dataset
print("--- Step 1: Loading and Preparing Data ---")
try:
    df = pd.read_csv('spam.csv', encoding='latin-1')
except FileNotFoundError:
    print("\n!!! ERROR: Please upload the 'spam.csv' file to the Colab environment first. !!!")
    exit()

df = df.iloc[:, [0, 1]]
df.columns = ['label', 'message']
df['label_num'] = df['label'].map({'spam': 1, 'ham': 0})
print("Data loaded successfully.")

# 4. Text Cleaning Function
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    words = text.split()
    words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words]
    return ' '.join(words)

df['cleaned_message'] = df['message'].apply(clean_text)
print("Text cleaning complete.")

# 5. Feature Extraction and Data Splitting (for statistical purposes)
tfidf_vectorizer = TfidfVectorizer(max_features=3000)
X = tfidf_vectorizer.fit_transform(df['cleaned_message']).toarray()
y = df['label_num'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratif

# 6. Train and Evaluate Models (for statistical purposes)
print("\n--- Step 2: Training Models to Generate Statistics ---")
# Decision Tree Model
```

```
# Decision Tree Model
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

# Random Forest Model
rf_model_for_stats = RandomForestClassifier(n_estimators=200, random_state=42)
rf_model_for_stats.fit(X_train, y_train)
rf_pred = rf_model_for_stats.predict(X_test)
print("Models trained successfully.")

# 7. Display Numerical Results and Visualizations
print("\n" + "="*60)
print("           MODEL PERFORMANCE RESULTS (FOR STATISTICS)")
print("="*60)

dt_accuracy = accuracy_score(y_test, dt_pred)
dt_precision = precision_score(y_test, dt_pred)
dt_recall = recall_score(y_test, dt_pred)
dt_f1 = f1_score(y_test, dt_pred)
print("--- Decision Tree Model ---")
print(f"Accuracy:  {dt_accuracy:.4f}")
print(f"Precision: {dt_precision:.4f}")
print(f"Recall:    {dt_recall:.4f}")
print(f"F1-Score:  {dt_f1:.4f}\n")

rf_accuracy = accuracy_score(y_test, rf_pred)
rf_precision = precision_score(y_test, rf_pred)
rf_recall = recall_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)
print("--- Random Forest Model ---")
print(f"Accuracy:  {rf_accuracy:.4f}")
print(f"Precision: {rf_precision:.4f}")
print(f"Recall:    {rf_recall:.4f}")
print(f"F1-Score:  {rf_f1:.4f}")
print("="*60)

# --- FIGURE 1: Data Distribution (Pie Chart) ---
plt.style.use('seaborn-v0_8-whitegrid')
plt.figure(figsize=(8, 6))
label_counts = df['label'].value_counts()
plt.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%', startangle=140, colors=['#66b
plt.title('Figure 1: Distribution of Ham vs. Spam in the Dataset', fontsize=14, fontweight='bold'
plt.ylabel('')
plt.show()

# --- FIGURE 2: Confusion Matrices ---
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Figure 2: Confusion Matrices for Both Models', fontsize=16, fontweight='bold')
sns.heatmap(confusion_matrix(y_test, dt_pred), annot=True, fmt='d', cmap='Blues', ax=axes[0], cba
axes[0].set_title('A) Decision Tree', fontsize=12)
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')
sns.heatmap(confusion_matrix(y_test, rf_pred), annot=True, fmt='d', cmap='Greens', ax=axes[1], cb
axes[1].set_title('B) Random Forest', fontsize=12)
axes[1].set_xlabel('Predicted Label')
axes[1].set_ylabel('True Label')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# --- FIGURE 3: Performance Comparison (Bar Chart) ---
metrics_data = { 'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'], 'Decision Tree': [dt_
metrics_df = pd.DataFrame(metrics_data)
metrics_df.set_index('Metric').plot(kind='bar', figsize=(12, 7), rot=0, colormap='viridis')
plt.title('Figure 3: Performance Comparison of DT and RF Models', fontsize=14, fontweight='bold')
plt.ylabel('Score')
plt.xlabel('')
plt.ylim(0.0, 1.1)
plt.legend(title='Algorithm')
plt.show()
```

```
# ----------------------------------------------------------------------
# --- PART 2: INTERACTIVE SYSTEM WITH REAL-TIME LEARNING MEMORY ---
# ----------------------------------------------------------------------

print("\n\n" + "="*60)
print("    Preparing the Interactive System (Training Final Model)")
print("="*60)

# Train the final model
final_model = DecisionTreeClassifier(random_state=42)
final_model.fit(X, y)
print("Final model trained successfully.")

# Create the Feedback Memory Dictionary (This stores your corrections!)
feedback_memory = {}

# Start the interactive system
print("\n" + "="*60)
print("    Spam Detection System is Ready for Use")
print("    (Featuring Real-Time Feedback Memory)")
print("="*60)

while True:
    try:
        user_input = input("\n> Enter a message to classify (or type 'exit' to quit): ")

        if user_input.lower() == 'exit':
            print("\nSystem shut down. Goodbye!")
            break

        if not user_input.strip():
            print("Please enter some text.")
            continue

        # 1. Clean the message
        cleaned_message = clean_text(user_input)

        # 2. Check if the system remembers this message from previous feedback
        if cleaned_message in feedback_memory:
            prediction = feedback_memory[cleaned_message]
            print("  🧠 [Memory Match]: The system remembered your previous correction for this m
        else:
            # If not in memory, use the Machine Learning Model
            vectorized_message = tfidf_vectorizer.transform([cleaned_message])
            prediction = final_model.predict(vectorized_message)[0]

        # 3. Display Classification
        if prediction == 1:
            predicted_label = "SPAM"
            opposite_label = "HAM (Not Spam)"
            opposite_code = 0
            print("  🚨 CLASSIFICATION: SPAM")
        else:
            predicted_label = "HAM (Not Spam)"
            opposite_label = "SPAM"
            opposite_code = 1
            print("  ✅ CLASSIFICATION: HAM (Not Spam)")

        # 4. User Evaluation Feedback (True / False)
        while True:
            feedback = input("  > Was this classification correct? (True/False): ").strip().lower

            if feedback in ['true', 't']:
                print("  [System Update]: Excellent! Model accuracy confirmed.")
                break

            elif feedback in ['false', 'f']:
                # Save the correction to memory so it never makes this mistake again!
```

```
                feedback_memory[cleaned_message] = opposite_code
                print("  [System Update]: Error acknowledged. Re-evaluating classification...")
                print(f"  🔄 FINAL CLASSIFICATION CORRECTED TO: {opposite_label}")
                print("  💾 [Saved to Memory]: The system has learned this correction for future
                break

            else:
                print("  [System Error]: Invalid input. Please type 'True' or 'False'.")

    except KeyboardInterrupt:
        print("\nSystem shut down. Goodbye!")
        break
```

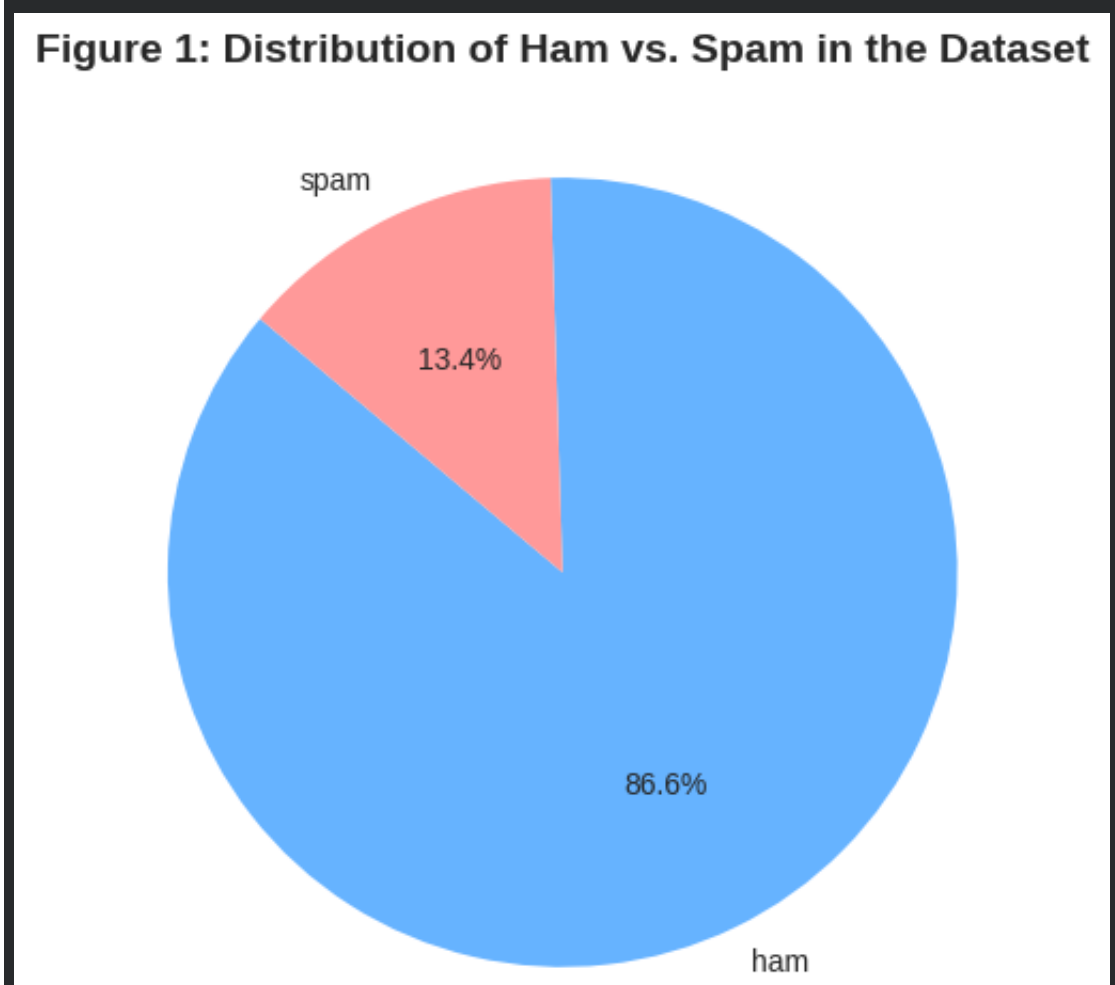
⌨

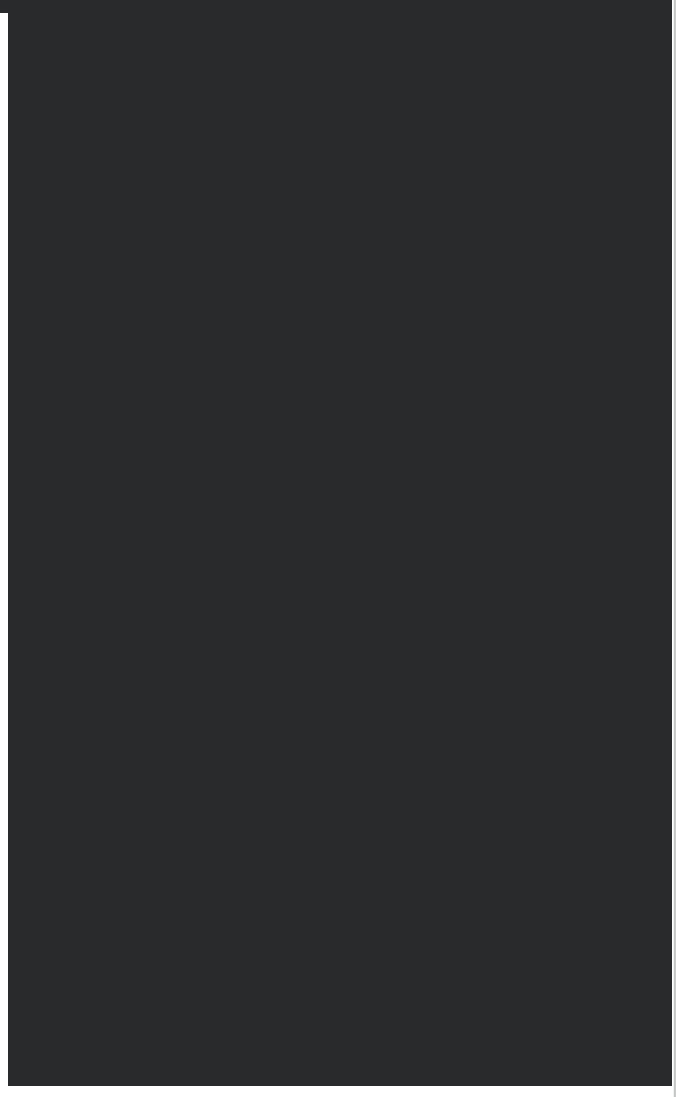
```
--- Step 1: Loading and Preparing Data ---
Data loaded successfully.
Text cleaning complete.

--- Step 2: Training Models to Generate Statistics ---
Models trained successfully.

============================================================
          MODEL PERFORMANCE RESULTS (FOR STATISTICS)
============================================================
--- Decision Tree Model ---
Accuracy:  0.9570
Precision: 0.8389
Recall:    0.8389
F1-Score:  0.8389

--- Random Forest Model ---
Accuracy:  0.9758
Precision: 0.9919
Recall:    0.8255
F1-Score:  0.9011
============================================================
```





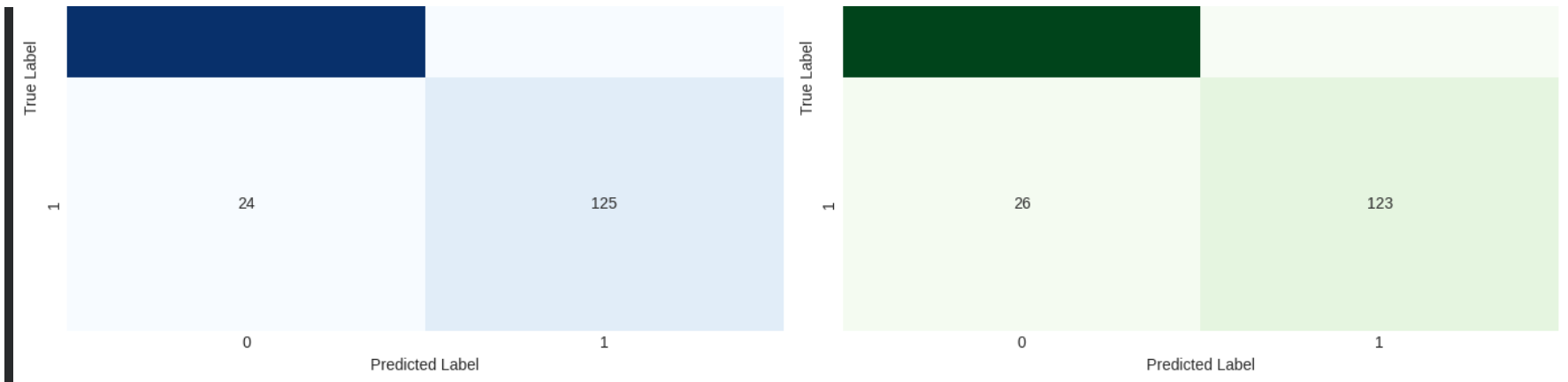
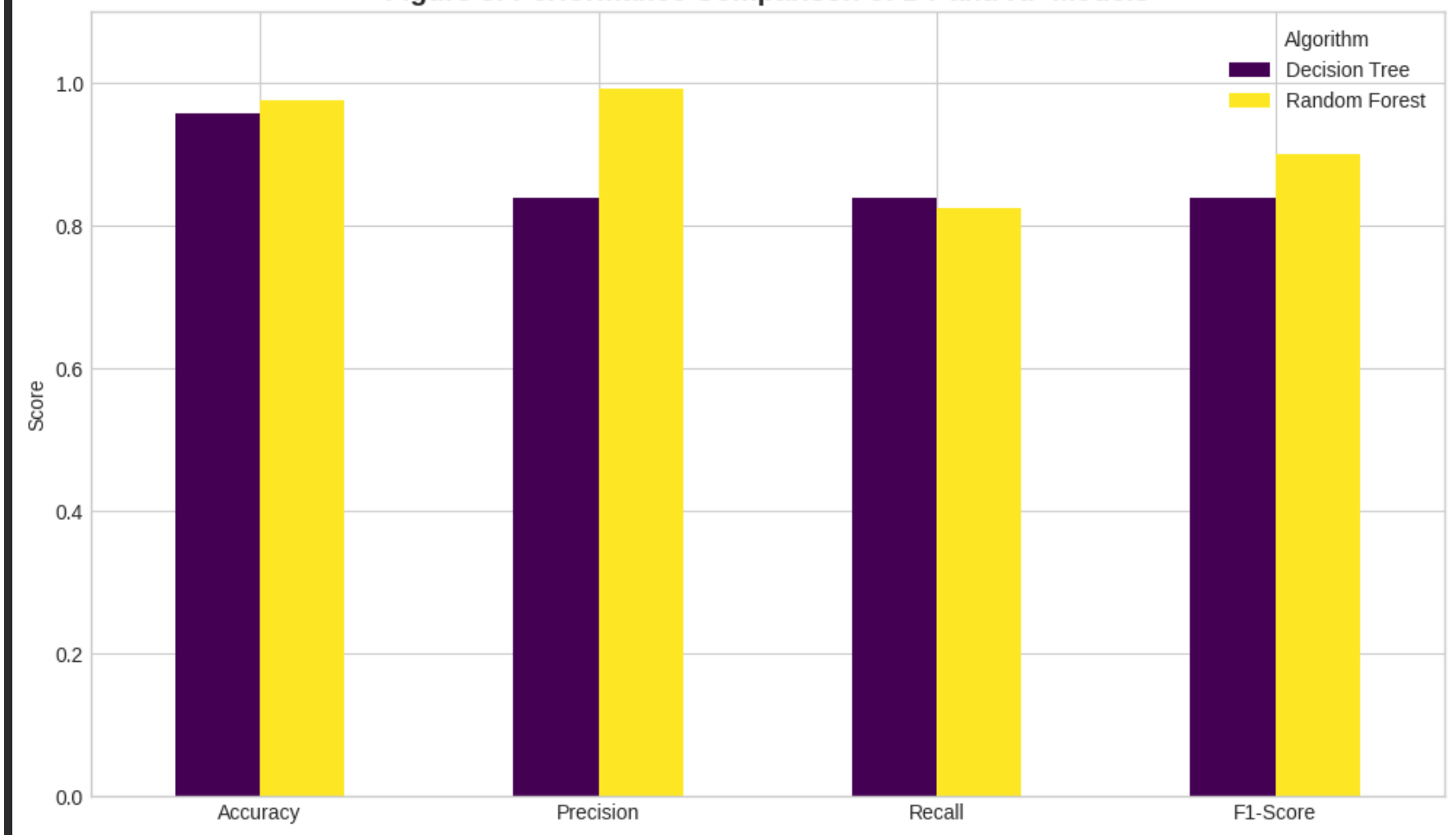


Figure 3: Performance Comparison of DT and RF Models




```
============================================================
    Preparing the Interactive System (Training Final Model)
============================================================
Final model trained successfully.

============================================================
    Spam Detection System is Ready for Use
    (Featuring Real-Time Feedback Memory)
============================================================

> Enter a message to classify (or type 'exit' to quit):
```